Experimental Details for "Learn When and Where to Connect: Adaptive Virtual Nodes for Dynamic Message Passing on Graphs"

Jaejun Lee
School of Computing
KAIST
Daejeon, Republic of Korea
jjlee98@kaist.ac.kr

Joyce Jiyoung Whang*
Department of AI Computing
KAIST
Daejeon, Republic of Korea
jjwhang@kaist.ac.kr

ACM Reference Format:
Jaejun Lee and Joyce Jiyoung Whang. 2026. Experimental Details for "Learn When and Where to Connect: Adaptive Virtual Nodes for Dynamic Message Passing on Graphs". In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3770855.3818013

This document provides experimental details of MAVN proposed in the paper "Learn When and Where to Connect: Adaptive Virtual Nodes for Dynamic Message Passing on Graphs" published in the Proceedings of the 32nd SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026).

1 Experimental Details of MAVN

In Tables 1, 2, and 3, we provide the best hyperparameter settings for each experiment, along with the number of parameters (#p), runtime (time), and GPU memory usage (Mem.). Note that the hyperparameters were selected based on the best validation performance. For the LRGB datasets [7], we report the runtime of a single run. Similarly, for the heterophilic graph datasets [18], we report the runtime for a single split. GPU memory usage was measured using PyTorch's `torch.cuda.max_memory_allocated()` function. Except for the *Tree-LeafCount* dataset, all experiments use a cosine learning rate scheduler [15] with linear warmup over e_{w} epochs, within a total of e_{t} training epochs. During the warmup stage, the Gumbel-Softmax temperature τ is held constant at τ_{i}. After warmup, τ decays exponentially to τ_{f} over e_τ epochs, and remains fixed at τ_{f} for the remainder of training. The learning rate reaches its maximum value, $\mathrm{lr}_{\max}$, immediately after the warmup phase, and then decays to 0 following the cosine schedule. For *Tree-LeafCount* dataset, we do not apply any scheduling to the learning rate or τ; instead, they are fixed at $\mathrm{lr}_{\max}$ and τ_{f}, respectively.

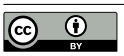
We denote the number of layers in the MLP used in Eq. 3 by L_{MLP}, and the number of layers in the MLP decoder by L_{dec}. PE indicates the type of positional encoding used. Norm. indicates the normalization technique. The dropout rate is denoted by δ, and the batch size is denoted by n_{b}. The dimension of node representations is fixed to d across all layers, except for certain heterophilic graph datasets where the first layer operates on raw input features. In these cases, we explicitly distinguish between d_0 for the first layer and d for subsequent layers. The hidden dimension of $\mathrm{MLP}^{(l)}$ matches the dimension of the node representations at the $(l-1)$-th layer. BN, LN, and GN refer to batch normalization [12], layer normalization [2], and graph normalization [4], respectively. ✗ indicates that no normalization is applied. Lap and RW refer to Laplacian positional encoding [5] and random walk structural encoding [6], respectively. We use $c_z^{(l)} = \mathrm{wm}$ to denote a weighted mean, and $c_z^{(l)} = \mathrm{ws}$ to denote a weighted sum.

Table 1 presents the best hyperparameter settings, the number of parameters, runtime, and GPU memory usage for MAVN with GCN [14] backbone on *Tree-LeafCount* [19] (*Tree-LC*) and *Tree-NeighborsMatch* [1] (*Tree-NM*) with depths ranging from 4 to 6. We use GCN implementation from [20]. Residual connections [9] are applied in all cases, $c_z^{(l)}$ is set to 1, and the normalization applied to MPNN is also applied to the input of the decoder. We use AdamW [16] as the optimizer, GELU [10] as the activation function, and the cross-entropy as the loss function. We use an NVIDIA RTX 2080 Ti GPU for *Tree-LeafCount* with depths 4 to 6 and *Tree-NeighborsMatch* with depth 4. For *Tree-NeighborsMatch* with depths 5 and 6, we use an NVIDIA RTX A6000 GPU. Positional encoding is not used in these experiments.

Table 2 shows the best hyperparameter settings, the number of parameters, runtime, and GPU memory usage for MAVN with GCN [14], GINE [11], and GatedGCN [3] backbones on *Peptides-func*, *Peptides-struct*, *PascalVOC-SP*, and *COCO-SP*. Residual connections are applied in all cases, and the normalization applied to MPNN is also applied to the input of the decoder. We use AdamW [16] as the optimizer and GELU [10] as the activation function. For the loss function, we use the binary cross-entropy for *Peptides-func*, mean absolute error for *Peptides-struct*, and cross-entropy for *PascalVOC-SP* and *COCO-SP*. Additionally, gradient clipping with a threshold of 1.0 is applied for *Peptides-func* and *Peptides-struct*. An NVIDIA RTX 3090 GPU is used for experiments on *Peptides-func* and *Peptides-struct*, while an NVIDIA RTX A6000 GPU is used for experiments on *PascalVOC-SP* and *COCO-SP*. Note that we run MAVN with four different seeds and report the mean and the standard deviation, following the standard evaluation protocol [7].

Table 3 shows the best hyperparameter settings, the number of parameters, runtime, and GPU memory usage for MAVN with GCN [14], GAT [21], and GraphSAGE [8] (GSAGE) backbones on *roman-empire*, *amazon-ratings*, *minesweeper*, *tolokers*, and *questions*. The "ver." column indicates which backbone implementation was used; either from [18] (denoted as #1) or [17] (denoted as #2). The

*Corresponding author.



KDD '26, Jeju Island, Republic of Korea

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3818013

Table 1: Best hyperparameter settings, the number of parameters (#p), runtime (time), and GPU memory usage (Mem.) for MAVN on two synthetic datasets, *Tree-LC* and *Tree-NM*.

	depth	d	d_{dot}	M	h	normalization MPNN	normalization MLP	normalization $\mathbf{k}_z^{(l)}$	L	L_{MLP}	L_{dec}	α	e_{w}	lr_{max}	δ	e_{t}	n_{b}	τ_{i}	τ_{f}	e_τ	#p	time	Mem.
Tree-LC	4	32	32	1	1	✗	✗	✗	1	1	2	1	N/A	1e-3	0	2000	4K	N/A	1	N/A	5K	3m	0.4GB
	5	32	32	1	1	✗	✗	✗	1	1	2	1	N/A	1e-3	0	2000	4K	N/A	1	N/A	6K	4m	0.9GB
	6	32	32	1	1	✗	✗	✗	1	1	2	1	N/A	1e-3	0	2000	4K	N/A	1	N/A	10K	27m	1.7GB
Tree-NM	4	32	32	4	1	✗	GN	LN	5	1	1	1	200	2e-3	0	2000	2K	10	1	1800	15K	35m	1.4GB
	5	32	32	32	1	✗	LN	LN	6	1	1	0	100	1e-3	0	1000	1K	10	1	900	30K	80m	7.4GB
	6	32	32	64	1	✗	LN	LN	7	1	1	0	200	3e-3	0	2000	1K	10	1	1800	52K	8h	32GB

Table 2: Best hyperparameter settings, the number of parameters (#p), runtime (time), and GPU memory usage (Mem.) for MAVN on the LRGB datasets.

	Peptides-func			*Peptides-struct*			*PascalVOC-SP*			*COCO-SP*		
	GCN	GINE	GatedGCN	GCN	GINE	GatedGCN	GCN	GINE	GatedGCN	GCN	GINE	GatedGCN
PE	Lap+RW	Lap+RW	Lap+RW	Lap	Lap	Lap	RW	None	None	None	None	None
d	245	130	120	183	158	141	178	140	90	223	191	104
d_{dot}	16	48	48	48	48	48	48	48	32	48	48	32
M	12	24	24	24	24	12	24	24	24	12	12	12
h	4	4	8	2	1	4	2	1	1	1	1	4
MPNN Norm.	BN	BN	BN	LN	BN	✗	LN	BN	BN	✗	LN	LN
MLP Norm.	GN	GN	GN	LN	LN	GN	GN	✗	GN	LN	GN	LN
$\mathbf{k}_z^{(l)}$ Norm.	LN	LN	LN	LN	LN	LN	✗	LN	✗	LN	LN	✗
L	6	8	6	6	6	4	10	10	10	8	6	8
L_{MLP}	1	2	1	2	2	1	1	1	1	1	1	1
L_{dec}	2	3	3	2	1	3	3	2	2	1	1	1
$c_z^{(l)}$	wm	wm	wm	wm	ws	wm	wm	wm	wm	wm	wm	wm
α	1.0	1.0	1.0	1.0	0.4	1.0	1.0	1.0	1.0	1.0	1.0	1.0
e_{w}	15	5	5	5	10	10	10	10	10	10	10	10
lr_{max}	2e-3	2e-3	2e-3	4e-3	2e-3	1e-3	4e-3	2e-3	3e-3	1e-3	1e-3	1e-3
δ	0.2	0.1	0.1	0.1	0.1	0.2	0.1	0.1	0.1	0.1	0.1	0.1
e_{t}	750	250	250	250	500	500	200	200	200	200	200	200
n_{b}	200	200	200	200	200	200	50	50	50	50	50	50
τ_{i}	10	10	10	10	10	10	10	10	10	10	10	10
τ_{f}	0.1	0.1	0.1	0.1	0.1	0.1	0.5	0.5	1.0	0.5	0.5	0.5
e_τ	300	100	100	100	200	200	100	100	100	100	100	100
#p	497K	548K	543K	549K	549K	494K	546K	545K	492K	549K	545K	497K
time	210m	90m	90m	70m	130m	110m	240m	150m	210m	44h	40h	48h
Mem.	7.7GB	8.6GB	12GB	8.3GB	8.4GB	6.5GB	14GB	6.0GB	14GB	7.6GB	6.0GB	11.1GB

implementation version to use was selected based on validation performance. In the "Loss" column, CE denotes the use of cross-entropy loss, while BCE indicates the binary cross-entropy loss. For implementations from [18], the normalization used in the MPNN is also applied to the decoder input, GELU [10] activation functions are employed, and the AdamW [16] optimizer is used. In contrast, implementations from [17] do not apply normalization to the decoder input, use ReLU activation functions, and employ the Adam [13] optimizer. Full-batch training is performed per epoch, with $\tau_i = 10.0$ and $\tau_f = 0.1$ for all datasets. We use an NVIDIA RTX 3090 GPU for *amazon-ratings*, an NVIDIA RTX 2080 Ti GPU for *minesweeper* and *tolokers*, and an NVIDIA RTX A6000 GPU for *questions*. On *roman-empire*, MAVN-GCN is trained using an NVIDIA RTX 3090 GPU, while MAVN-GAT and MAVN-GraphSAGE use an NVIDIA RTX 2080 Ti GPU.

Table 3: Best hyperparameter settings, the number of parameters (#p), runtime (time), and GPU memory usage (Mem.) for MAVN on the Heterophilic Graph datasets.

	roman-empire			*amazon-ratings*			*minesweeper*			*tolokers*			questions		
	GCN	GAT	GSAGE	GCN	GAT	GSAGE	GCN	GAT	GSAGE	GCN	GAT	GSAGE	GCN	GAT	GSAGE
ver.	#2	#1	#2	#2	#2	#2	#2	#2	#2	#2	#1	#2	#2	#1	#2
d_0	512	128	256	300	300	300	7	7	7	10	32	10	512	128	512
d	512	128	256	512	512	512	64	64	64	32	32	32	512	128	512
d_{dot}	8	128	8	8	16	4	64	128	64	48	96	64	8	16	4
M	16	8	16	24	24	16	40	20	80	24	72	72	12	24	16
h	1	8	1	2	1	1	2	8	1	1	4	1	1	2	1
MPNN Norm.	BN	LN	BN	BN	BN	BN	BN	BN	BN	LN	LN	✗	✗	LN	LN
MLP Norm.	GN	✗	GN	✗	GN	LN	GN	BN	GN	LN	LN	✗	GN	GN	BN
$\mathbf{k}_z^{(l)}$ Norm.	LN	✗	N	✗	LN	LN	✗	✗	✗	✗	LN	LN	LN	✗	LN
L	9	9	9	4	4	9	12	15	15	4	6	8	10	10	6
L_{MLP}	1	2	1	1	1	1	2	1	1	2	1	1	1	1	1
L_{dec}	1	2	1	1	1	1	1	1	1	1	1	1	1	2	2
$c_z^{(l)}$	wm	ws	wm	wm	wm	wm	wm	wm	wm	wm	ws	wm	ws	ws	wm
α	0.3	0.5	0.4	0.2	0.2	0.2	0.2	0.5	0.2	0.1	0.1	0.1	0.5	0.2	0.3
e_{w}	250	150	250	250	300	250	200	300	300	250	100	250	250	50	150
lr_{max}	2e-2	2e-2	2e-2	2e-2	4e-3	5e-3	5e-2	4e-2	5e-2	2e-2	1e-2	1e-2	1e-4	4e-2	5e-5
δ	0.5	0.4	0.3	0.5	0.5	0.5	0.2	0.2	0.2	0.5	0.4	0.3	0.4	0.3	0.1
e_{t}	2500	3000	2500	2500	3000	2500	2000	3000	3000	2500	2000	2500	2500	1000	1500
e_τ	1250	1500	1250	1250	1500	1250	1000	1500	1500	1250	1000	1250	1250	500	750
Loss	CE	CE	CE	CE	CE	CE	CE	CE	CE	CE	BCE	CE	BCE	BCE	BCE
#p	5.1M	828K	1.3M	2.0M	2.0M	6.9M	255K	300K	392K	25K	97K	95K	5.6M	619K	3.7M
time	30m	20m	20m	20m	33m	46m	25m	30m	60m	12m	14m	30m	72m	17m	24m
Mem.(GB)	15.1	4.0	7.0	7.7	14.5	18.2	3.9	3.5	8.4	2.1	7.9	6.2	34.5	20.6	17.4

References

[1] Uri Alon and Eran Yahav. 2021. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *Proceedings of the 9th International Conference on Learning Representations*.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. *arXiv preprint arXiv.1607.06450* (2016). doi:10.48550/arXiv.1607.06450

[3] Xavier Bresson and Thomas Laurent. 2017. Residual Gated Graph ConvNets. *arXiv preprint arXiv.1711.07553* (2017). doi:10.48550/arXiv.1711.07553

[4] Tianle Cai, Shengjie Luo, Keyulu Xu, Di He, Tie-Yan Liu, and Liwei Wang. 2021. GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training. In *Proceedings of the 38th International Conference on Machine Learning*. 1204–1215.

[5] Vijay Prakash Dwivedi, Chaitanya K. Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2023. Benchmarking Graph Neural Networks. *Journal of machine learning research* 24, 43 (2023), 1–48.

[6] Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2022. Graph Neural Networks with Learnable Structural and Positional Representations. In *Proceedings of the 10th International Conference on Learning Representations*.

[7] Vijay Prakash Dwivedi, Ladislav Rampášek, Michael Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. 2022. Long Range Graph Benchmark. In *Proceedings of the 36th Conference on Neural Information Processing Systems*. 22326–22340.

[8] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Proceedings of the 31st Conference on Neural Information Processing Systems*. 1025–1034.

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.

[10] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv.1606.08415* (2016). doi:10.48550/arXiv.1606.08415

[11] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2020. Strategies for Pre-training Graph Neural Networks. In *Proceedings of the 8th International Conference on Learning Representations*.

[12] Sergey Ioffe and Christian Szegedy. 2015. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *Proceedings of the 32nd International Conference on Machine Learning*. 448–456.

[13] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations*.

[14] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proceedings of the 5th International Conference on Learning Representations*.

[15] Ilya Loshchilov and Frank Hutter. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. In *Proceedings of the 5th International Conference on Learning Representations*.

[16] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *Proceedings of the 7th International Conference on Learning Representations*.

[17] Yuankai Luo, Lei Shi, and Xiao-Ming Wu. 2024. Classic GNNs are Strong Baselines: Reassessing GNNs for Node Classification. In *Proceedings of the 38th Conference on Neural Information Processing Systems*. 97650–97669.

[18] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A critical look at the evaluation of GNNs under heterophily: Are we really making progress?. In *Proceedings of the 11th International Conference on Learning Representations*.

[19] Chendi Qian, Andrei Manolache, Kareem Ahmed, Zhe Zeng, Guy Van den Broeck, Mathias Niepert, and Christopher Morris. 2024. Probabilistically Rewired Message-Passing Neural Networks. In *Proceedings of the 12th International Conference on Learning Representations*.

[20] Jan Tönshoff, Martin Ritzert, Eran Rosenbluth, and Martin Grohe. 2024. Where Did the Gap Go? Reassessing the Long-Range Graph Benchmark. *Transactions on Machine Learning Research* (2024).

[21] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the 6th International Conference on Learning Representations*.